

TASK 26: ASSIGNMENT NO. 26

Name of Student : Anuja Vilas Wankhade

Domain: Artificial intelligence and machine Learning (AIML)

College Name: Manav School of Polytechnic AKOLA

Branch : Computer Engineering

GitHub Repository: <https://github.com/anujaw193-star/python-project-collection>

1. (Understanding & Dataset Creation)

Code:

```
from sklearn.datasets import make_blobs
import pandas as pd

# Create synthetic dataset
X, y = make_blobs(n_samples=500, centers=4, cluster_std=0.70, random_state=42)

# Convert to DataFrame
df = pd.DataFrame(X, columns=['feature_1', 'feature_2'])

# Display first 10 rows
print("First 10 rows:")
print(df.head(10))

# Display shape
print("\nShape of DataFrame:")
print(df.shape)
```

Output:

```
... First 10 rows:
   feature_1  feature_2
0  -8.033836   6.831299
1  -8.767448  -6.918116
2   6.172741   1.407361
3  -2.783673   7.989826
4  -6.038777  -7.165762
5  -6.298392  -7.479368
6  -7.046269   7.256281
7  -6.690307  -6.915276
8   3.892545   1.873504
9  -7.317000  -6.861846

Shape of DataFrame:
(500, 2)
```

2. (Data Scaling)

Code:

```
[3]
✓ Os
from sklearn.preprocessing import StandardScaler
import pandas as pd
from sklearn.datasets import make_blobs

# (Re-creating dataset from Q1 for continuity)
X, y = make_blobs(n_samples=500, centers=4, cluster_std=0.70, random_state=42)

# Apply Standard Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Show first 5 rows of scaled data
df_scaled = pd.DataFrame(X_scaled, columns=['feature_1', 'feature_2'])
print("First 5 rows of scaled data:")
print(df_scaled.head(5))
```

Output:

```
... First 5 rows of scaled data:
   feature_1  feature_2
0  -0.888474  0.629907
1  -1.028908 -1.562875
2   1.831069 -0.235113
3   0.116557  0.814671
4  -0.506563 -1.602370
```

3. (Basic K-Means Clustering)

Code:

```
[4]
✓ Os
from sklearn.cluster import KMeans

[5]
✓ Os
# Apply KMeans with n_clusters=3
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(X_scaled)

# Predict cluster labels and add as new column
df['cluster'] = kmeans.labels_

# Display first 10 rows
print("First 10 rows of updated DataFrame:")
print(df.head(10))
```

Output:

```
... First 10 rows of updated DataFrame:
   feature_1  feature_2  cluster
0  -8.033836  6.831299        1
1  -8.767448 -6.918116        2
2   6.172741  1.407361        0
3  -2.783673  7.989826        1
4  -6.038777 -7.165762        2
5  -6.298392 -7.479368        2
6  -7.046269  7.256281        1
7  -6.690307 -6.915276        2
8   3.892545  1.873504        0
9  -7.317000 -6.861846        2
```

4. (Visualizing Clusters)

Code:

```
[6] 1s ▶ import seaborn as sns
import matplotlib.pyplot as plt

df = pd.DataFrame(X_scaled, columns=['feature_1', 'feature_2'])
kmeans = KMeans(n_clusters=3, random_state=42)
df['Cluster'] = kmeans.fit_predict(X_scaled)

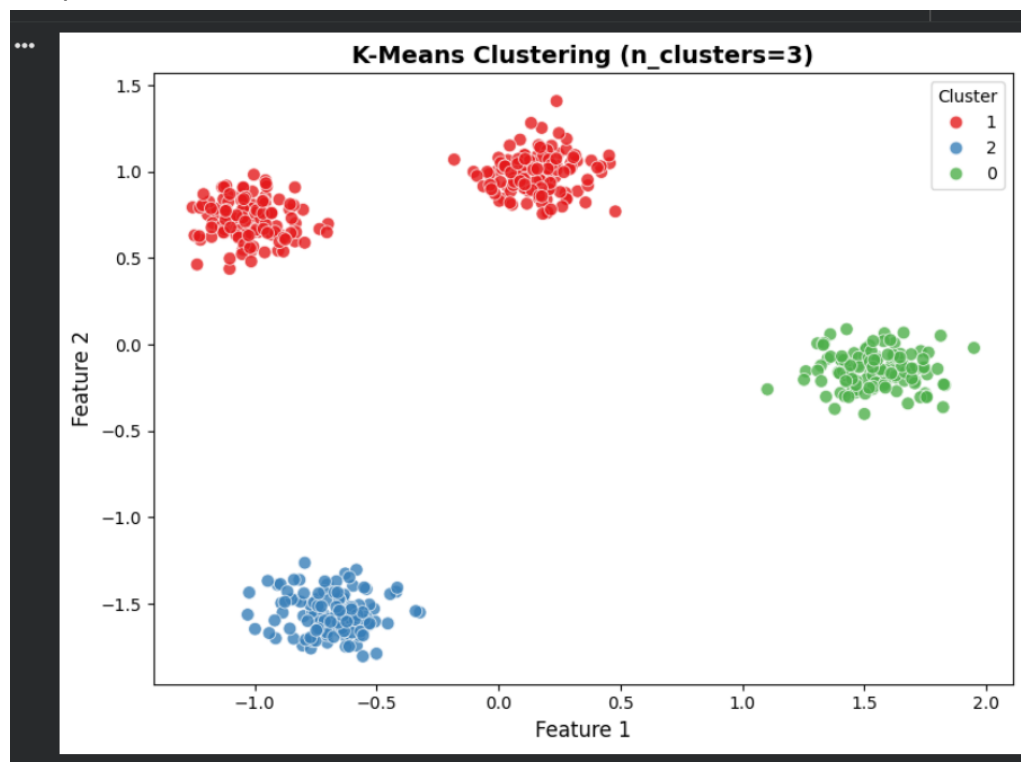
# Convert Cluster to string for better legend display
df['Cluster'] = df['Cluster'].astype(str)

# Create a scatter plot
plt.figure(figsize=(8, 6))

sns.scatterplot(
    data=df,
    x='feature_1',
    y='feature_2',
    hue='Cluster',
    palette='Set1',
    s=60,
    alpha=0.8
)

# Add title and labels
plt.title('K-Means Clustering (n_clusters=3)', fontsize=14, fontweight='bold')
plt.xlabel('Feature 1', fontsize=12)
plt.ylabel('Feature 2', fontsize=12)
plt.legend(title='Cluster', fontsize=10)
plt.tight_layout()
plt.show()
```

Output:



5. (Elbow Method – Finding Optimal K)

Code:

```
[7] 0s # Calculate inertia for K = 1 to 10
inertia = []
k_values = range(1, 11)

for k in k_values:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X_scaled)
    inertia.append(kmeans.inertia_)

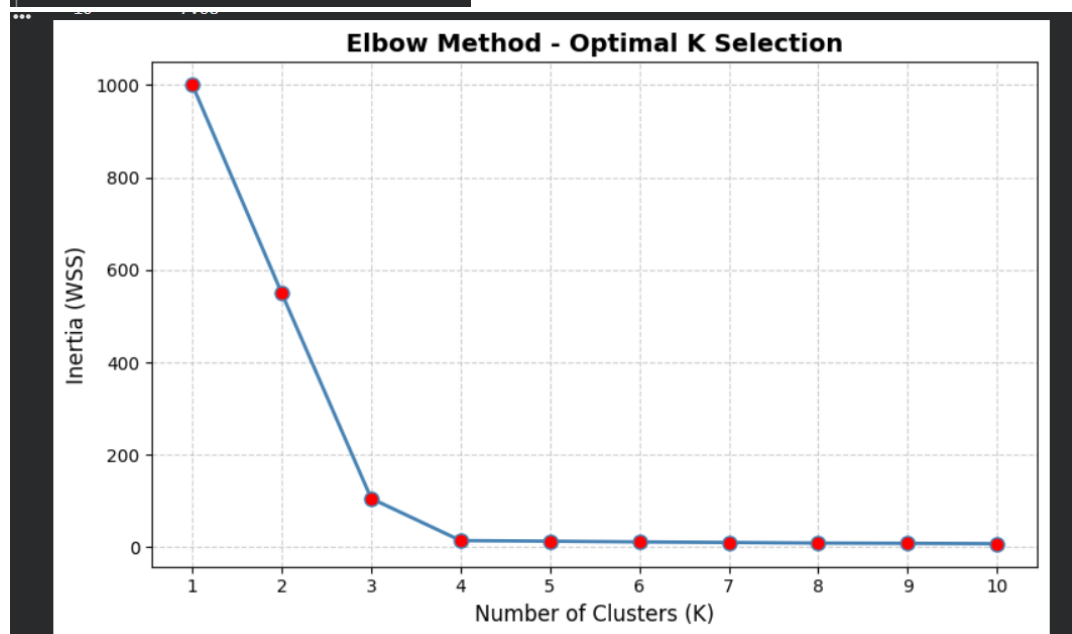
# Print K vs Inertia values
print(f'{K':>4} {'Inertia':>12}")
print('-' * 18)
for k, val in zip(k_values, inertia):
    print(f'{k:4d} {val:12.2f}")

# Plot the Elbow Graph
plt.figure(figsize=(8, 5))
plt.plot(k_values, inertia, marker='o', color='steelblue',
         linewidth=2, markersize=8, markerfacecolor='red')

plt.title('Elbow Method - Optimal K Selection', fontsize=14, fontweight='bold')
plt.xlabel('Number of Clusters (K)', fontsize=12)
plt.ylabel('Inertia (WSS)', fontsize=12)
plt.xticks(k_values)
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```

Output:

```
...      K      Inertia
-----
      1    1000.00
      2     549.07
      3     105.45
      4       14.36
      5       13.07
      6       11.48
      7       10.30
      8        8.95
      9        8.41
     10        7.68
```



6. (Final K-Means Model)

Code:

```
# Train the final KMeans model with the optimal number of clusters
kmeans = KMeans(n_clusters=4, random_state=42)

# Predict cluster labels
df['cluster'] = kmeans.fit_predict(df[['feature_1', 'feature_2']])

# Display first few rows with cluster labels
print(df.head())

# Print the number of data points in each cluster
print("\nNumber of data points in each cluster:")
print(df['cluster'].value_counts().sort_index())
```

Output:

```
...    feature_1  feature_2  Cluster
0    -0.888474    0.629907         1
1    -1.028908   -1.562875         2
2     1.831069   -0.235113         0
3     0.116557    0.814671         3
4    -0.506563   -1.602370         2

Number of data points in each cluster:
Cluster
0      125
1      125
2      125
3      125
Name: count, dtype: int64
```

7. (Visualizing Final Clusters)

Code:

```
[14] ✓ 3s # Create scatter plot
plt.figure(figsize=(8, 6))

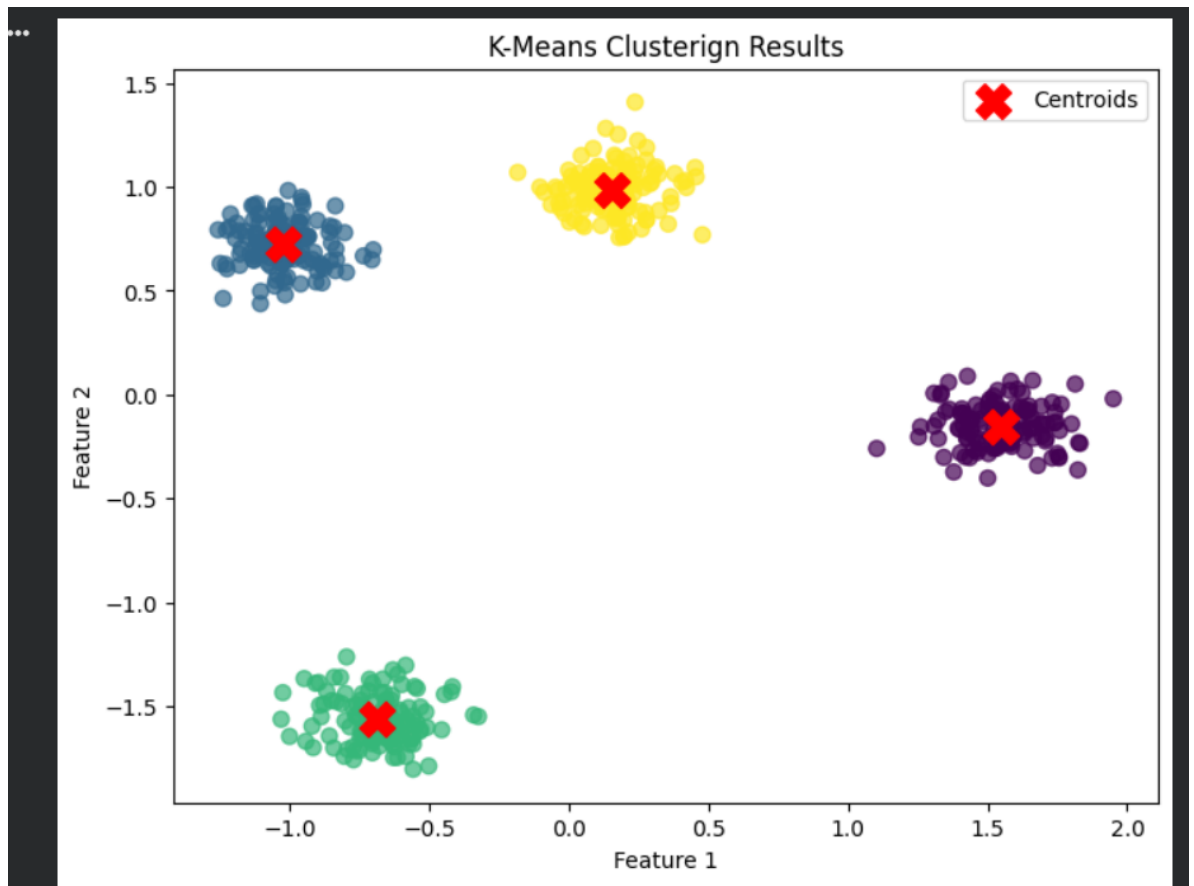
plt.scatter(
    df['feature_1'],
    df['feature_2'],
    c=df['cluster'],
    cmap='viridis',
    s=50,
    alpha=0.7
)

# Plot cluster centroids
centroids = kmeans.cluster_centers_
plt.scatter(
    centroids[:, 0],
    centroids[:, 1],
    c='red',
    s=250,
    marker='x',
    label='Centroids'
)

# Labels and title
plt.title("K-Means Clusterign Results")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.legend()

plt.show()
```

Output:



8. (Real Dataset Application)

Code:

```
[15]
✓ 0s #Q8. Real Dataset Application

import pandas as pd
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

# Load Iris dataset
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)

# Select numerical features
X = df.select_dtypes(include=['number'])

# Scale the data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Apply K-Means clustering
kmeans = KMeans(n_clusters=3, random_state=42)
df['Cluster'] = kmeans.fit_predict(X_scaled)

# Display first few rows
print(df.head())

# Show number of data points in each cluster
print("\nCluster Counts:")
print(df['Cluster'].value_counts().sort_index())
```

Output:

```
✓ ...      sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0          5.1             3.5             1.4             0.2
1          4.9             3.0             1.4             0.2
2          4.7             3.2             1.3             0.2
3          4.6             3.1             1.5             0.2
4          5.0             3.6             1.4             0.2

      Cluster
0           1
1           2
2           2
3           2
4           1

Cluster Counts:
Cluster
0     96
1     33
2     21
Name: count, dtype: int64
```

9. (Elbow Method on Real Dataset)

Code:

```
[16] ✓ Os # Elbow Method
      wcss = []

      for k in range(1, 11):
          kmeans = KMeans(n_clusters=k, random_state=42)
          kmeans.fit(X_scaled)
          wcss.append(kmeans.inertia_)

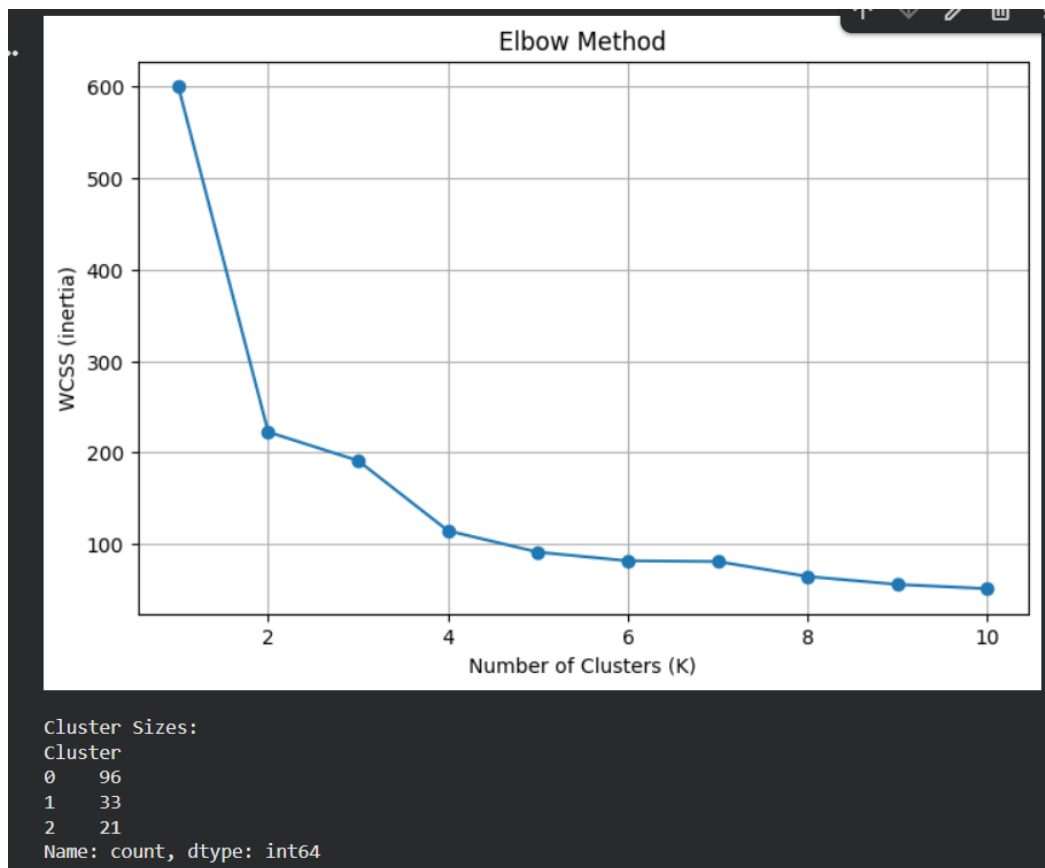
      # Plot the Elbow Graph
      plt.figure(figsize=(8, 5))
      plt.plot(range(1, 11), wcss, marker='o')
      plt.title('Elbow Method')
      plt.xlabel('Number of Clusters (K)')
      plt.ylabel('WCSS (inertia)')
      plt.grid(True)
      plt.show()

      # Final KMeans Model (Optimal K = 3)
      optimal_k = 3

      kmeans = KMeans(n_clusters=optimal_k, random_state=42)
      df['Cluster'] = kmeans.fit_predict(X_scaled)

      # Print cluster sizes
      print("\nCluster Sizes:")
      print(df['Cluster'].value_counts().sort_index())
```

Output:



10. (Mini Project – Complete Clustering Pipeline)

Code:

```
[19]
✓ 0s
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

df = pd.read_csv("Mall_Customers.csv")

print("First 5 Rows:")
print(df.head())

print("\nDataset Shape:")
print(df.shape)

print(df.info())

print(df.isnull().sum())

X = df[['Annual Income (k$)', 'Spending Score (1-100)']]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

wcss = []

for k in range(1,11):
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)
```



```
[19] 0s plt.figure(figsize=(8, 5))
plt.plot(range(1, 11), wcss, marker='o')
plt.title('Elbow Method')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('WCSS')
plt.grid(True)
plt.show()

optimal_k = 5

kmeans = KMeans(n_clusters=optimal_k, random_state=42)
df['Cluster'] = kmeans.fit_predict(X_scaled)

print("\nCluster Sizes:")
print(df['Cluster'].value_counts().sort_index())

plt.figure(figsize=(8,6))

plt.scatter(
    X_scaled[:, 0],
    X_scaled[:, 1],
    c=df['Cluster'],
    cmap='viridis',
    s=60
)
```

```
)

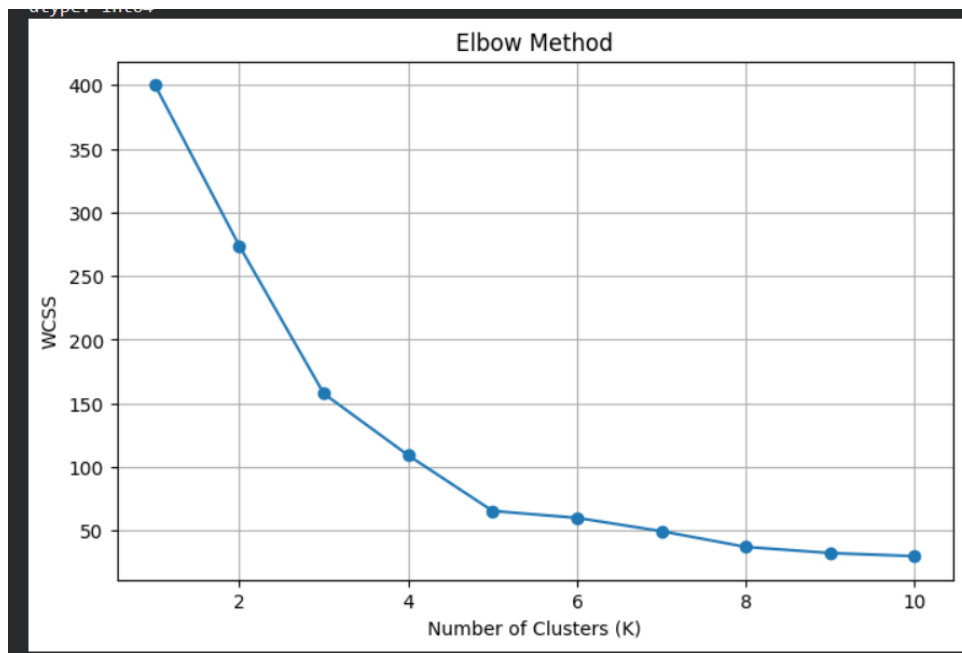
centroids = kmeans.cluster_centers_
plt.scatter(
    centroids[:, 0],
    centroids[:, 1],
    c='red',
    marker='x',
    s=250,
    label='Centroids'
)

plt.title("Customer Segmentation using K-Means")
plt.xlabel("Annual Income (Scaled)")
plt.ylabel("Spending Score (Scaled)")
plt.legend()
plt.grid(True)
plt.show()
```

Output:

```
... First 5 Rows:
  CustomerID  Genre  Age  Annual Income (k$)  Spending Score (1-100)
0           1   Male   19                15                39
1           2   Male   21                15                81
2           3  Female   20                16                 6
3           4  Female   23                16                77
4           5  Female   31                17                40

Dataset Shape:
(200, 5)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   CustomerID            200 non-null   int64
 1   Genre                 200 non-null   object
 2   Age                   200 non-null   int64
 3   Annual Income (k$)    200 non-null   int64
 4   Spending Score (1-100) 200 non-null   int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
None
CustomerID      0
Genre           0
Age             0
Annual Income (k$) 0
Spending Score (1-100) 0
dtype: int64
```



```

Cluster Sizes:
Cluster
0      81
1      39
2      22
3      35
4      23
Name: count, dtype: int64

```

